



University of Luxembourg

Networks Project

BINFO Networks class

Raphael Schmitz (022064850A) & Elyot Hamon (0220813006)
1-11-2024

Contents

1.	General architecture	2
1.1	Bootstrapping	2
1.2	Packet loss	2
1.3	Protocol description	2
1.3.1	Verifying data integrity.....	3
1.3.2	Details on savings in bandwidth and delivery time.....	4
1.3.3	Parameter choice	4
2.	Performance evaluation.....	4
2.1	2 clients downloading a 1GB file	4
2.2	10 clients downloading a 50 MB file.....	5
2.3	Observations.....	6
3.	Wireshark screenshots.....	6
3.1	1 GB + 2 Clients	6
3.2	50MB + 10 Clients.....	6
4.	Conclusion.....	7
5.	References	8

1. General architecture

1.1 Bootstrapping

To ensure that the server only starts sending the file once all clients have joined the server, we made sure that in our `start()` method for the server, the server first executes the `waitForJoin()` method.

This method works by using a while loop which checks if the map which contains the user addresses and ports is smaller than the number of total processes. In the loop it first calls the socket to receive a packet, which we transform it to a String and save the message in a variable. We then check if the first part of that message equals `JOIN`. If that's the case, we save all the parameters that are included. After that we also send an acknowledgement packet back to the client to ensure that everything's working properly.

On the client side it's relatively simple as we only call the `requestJoin()` method. This method consists of sending one packet which contains a message of following scheme:

```
JOIN filename totalProcesses probability windowSize timeout
```

After the method waits to receive an acknowledgement of the server.

As a note to the bootstrapping, it does not contain any simulation for packet loss, because we couldn't get it to run reliably in time every time.

1.2 Packet loss

To simulate packet loss, we created a wrapper method to send packets to a certain address and port. In this method we first generate a random float between 0 and 1 using `Math.random()` and then check if the number we got bigger than the probability. This ensures that a packet is sent successfully with a chance of (1-probability). This method is implemented both for the client and the server.

1.3 Protocol description

As it was optional to use the Selective-Repeat protocol, we chose only to implement the Go-Back N protocol.

The entire protocol is launched in a method called `sendFile()`.

As for the protocol, we start off by making sure that both the base and the nextSeq are set to 0. Then we retrieve our first packet by calling `createPacket(n)` for $n = 0$ and storing its return value in a variable called `nextPacket`. This method retrieves the n th packet of the file. After that the method enters a while loop in which we check that the next packet is not empty.

Once in the loop, we enter another while loop which sends all the packets in the window and increases nextSeq at the same time. If a new packet is null i.e. We reached the end of the file before the end of the window, we break out of this loop.

After this loop we call the `waitForAck()` method. In this method we first generate a copy of the client list to keep an eye on which users are already acknowledged. Then we also set the timeout for the socket along with starting a loop which checks that the list isn't empty.

In the execution part of this loop, we receive a packet and check if it's a valid ACK message and making sure that the `ackSeq` is in the window. If that's the case, we remove the client in the list.

In the case of a timeout, we resend each packet of the window to all unacknowledged users.

At the end of the method we also increment the base.

Once the entire file has been sent every client receives a packet declaring the end of the file.

On the client side, we receive a new packet, check if the sequence number is the same as the receiving base, write the packet data into the file, send an acknowledgement, and increment the receiving base. Also, if the sequence number is smaller than the receiving base, the client sends an acknowledgement of the sequence number, because an acknowledgement has been lost on the client side. This is done until the client receives a packet declaring the end of the file.

1.3.1 Verifying data integrity

To verify data integrity, the server adds a checksum at the front of the packet. The client checks if the checksum is incorrect and breaks out of the loop if that's the case.

For simplicity reasons, the checksum is calculated based on the length of the packet data.

In practice, this works quite well as we didn't observe any data corruption when testing it with different txt and mp4 files.

1.3.2 Details on savings in bandwidth and delivery time

In order to save on both bandwidth and delivery time, in our experience it was better to reduce the window size to a small number, so that in the case of a packet loss, you need to send less packets again.

The timeout also plays an important role, as in our experience the smaller the timeout the better, because there were always some issues after the first window is sent, as the client receives a new packet before he can send an acknowledgement of the packet beforehand.

1.3.3 Parameter choice

As for the parameter choices, we're quite open as we can change them freely when we start the application.

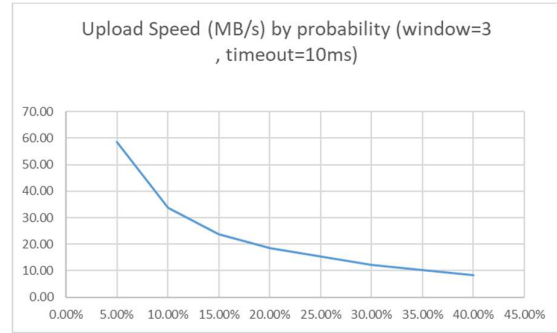
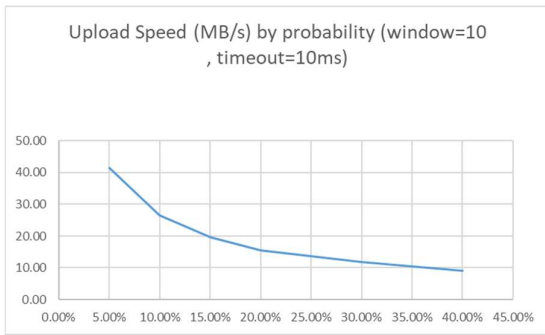
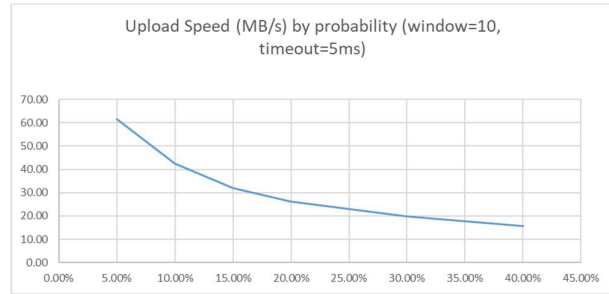
So we didn't really go for one parameter choice, but instead we can try out different parameters and make different observations.

The only parameters that are fixed are the server port, which is fixed at 6666 and the packet size which is fixed at 65507 bytes.

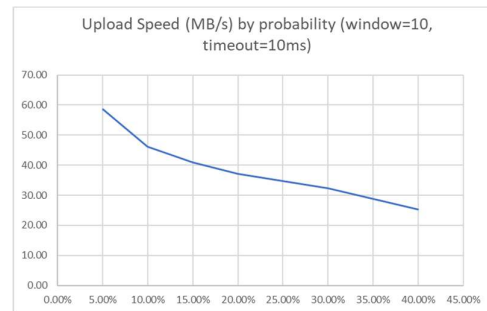
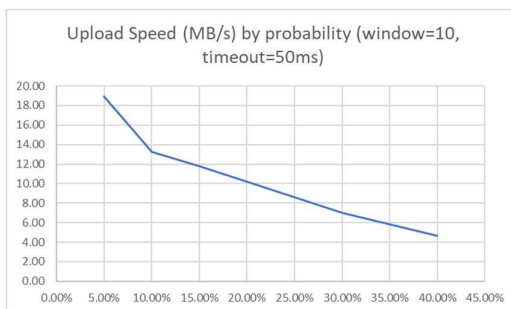
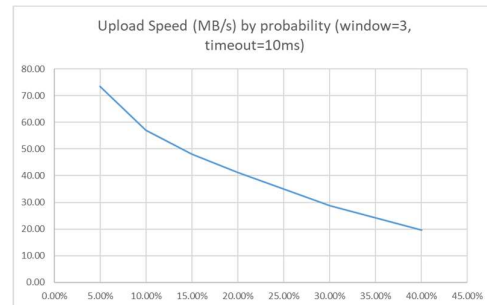
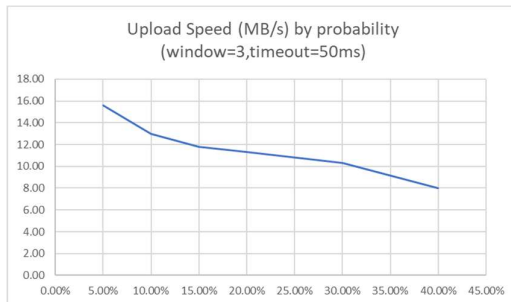
2. Performance evaluation

In the following we will analyse how different probabilities affect the bandwidth also switching up the window size and timeout. We didn't include the delay of the transfer as it can be calculated easily using the bandwidth and the description.

2.1 2 clients downloading a 1GB file



2.2 10 clients downloading a 50 MB file



2.3 Observations

By analysing the graphs, it becomes clear that overall, a lower timeout performs better.

However, when analysing the window size, it becomes clear that a small window size even though it may be better than a large window size at low probabilities, it drops more and even performs worse in some cases at 40% packet loss.

3. Wireshark screenshots

3.1 1 GB + 2 Clients

Address	Port	Packets	Bytes	Total Packets	Percent Filtered	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
127.0.0.1	6666	69,629	2 GB	69,629	100.00%	35,682	2 GB	33,947	1 MB
127.0.0.1	52906	34,547	1 GB	34,547	100.00%	17,102	690 kB	17,445	1 GB
127.0.0.1	52907	35,082	1 GB	35,082	100.00%	16,845	679 kB	18,237	1 GB

3.2 50MB + 10 Clients

127.0.0.1	6666	37,679	1 GB	37,679	100.00%	20,279	1 GB	17,400	676 kB
127.0.0.1	52117	7	5 kB	7	100.00%	7	5 kB	0	0 bytes
127.0.0.1	54601	4	4 kB	4	100.00%	4	4 kB	0	0 bytes
127.0.0.1	62734	3,592	127 MB	3,592	100.00%	1,649	64 kB	1,943	127 MB
127.0.0.1	62735	3,647	129 MB	3,647	100.00%	1,672	65 kB	1,975	129 MB
127.0.0.1	62736	3,951	139 MB	3,951	100.00%	1,835	71 kB	2,116	139 MB
127.0.0.1	62737	3,603	126 MB	3,603	100.00%	1,680	65 kB	1,923	126 MB
127.0.0.1	62738	3,746	131 MB	3,746	100.00%	1,750	68 kB	1,996	131 MB
127.0.0.1	62739	3,826	135 MB	3,826	100.00%	1,769	69 kB	2,057	135 MB
127.0.0.1	62740	3,878	136 MB	3,878	100.00%	1,796	70 kB	2,082	136 MB
127.0.0.1	62741	3,797	134 MB	3,797	100.00%	1,748	68 kB	2,049	134 MB
127.0.0.1	62742	3,922	139 MB	3,922	100.00%	1,797	70 kB	2,125	139 MB
127.0.0.1	62743	3,717	132 MB	3,717	100.00%	1,704	66 kB	2,013	132 MB

4. Conclusion

In conclusion, although there were areas where the protocol's design could have been optimized, we successfully developed a robust transport protocol leveraging the inherent features of UDP. Notably, our protocol bears resemblance to TCP in several aspects. Its reliability, a paramount criterion for any transport protocol, has been effectively addressed. One noteworthy advantage lies in the seamless file exchange capability between two independent PCs, with distinct server and client functionalities operating autonomously. This implementation not only meets our project goals but also translates our accomplishments and insights into a tangible and practical use case.

5. References

- <https://www.geeksforgeeks.org/java-nio-bytebuffer-class-in-java/>
- <https://docs.oracle.com/javase/8/docs/api/java/net/DatagramSocket.html>
- <https://docs.oracle.com/javase/8/docs/api/java/io/RandomAccessFile.html>
- <https://www.javatpoint.com/go-back-n-arq>
- <https://www.geeksforgeeks.org/java-net-datagramsocket-class-java/>
- https://gaia.cs.umass.edu/kurose_ross/ppt.php
- <https://www.scaler.com/topics/computer-network/go-back-n-arq/>